센서에서 주행까지

2026 ORDA 1 회차 멘토링 · 2026 년 10 월 29 일 오후 5 시

대회 자료로 만든 시뮬레이터
한 파일로 읽는 자율주행 예제

멘토 오상영

함께할 분 박소민 · 이혁준 · 양유경 · 추정윤

먼저 , 어떤 규칙을 구현했나요 ?

최신 첨부 규정집 (2026.10.08) 을 우선하고 9 월 미션 초안과 차이를 구분했습니다 .

코드에 반영한 기준

기본 주행 0.3~0.4 m/s 권고

서행 : 기본 속도의 1/2, 최대 0.2 m/s

보행자 : 완전 정지 후 3 초 이상 대기

정지 판단 기준은 차량의 앞바퀴

실제로 측정해야 하는 기준

정지선 오차 ±5 cm

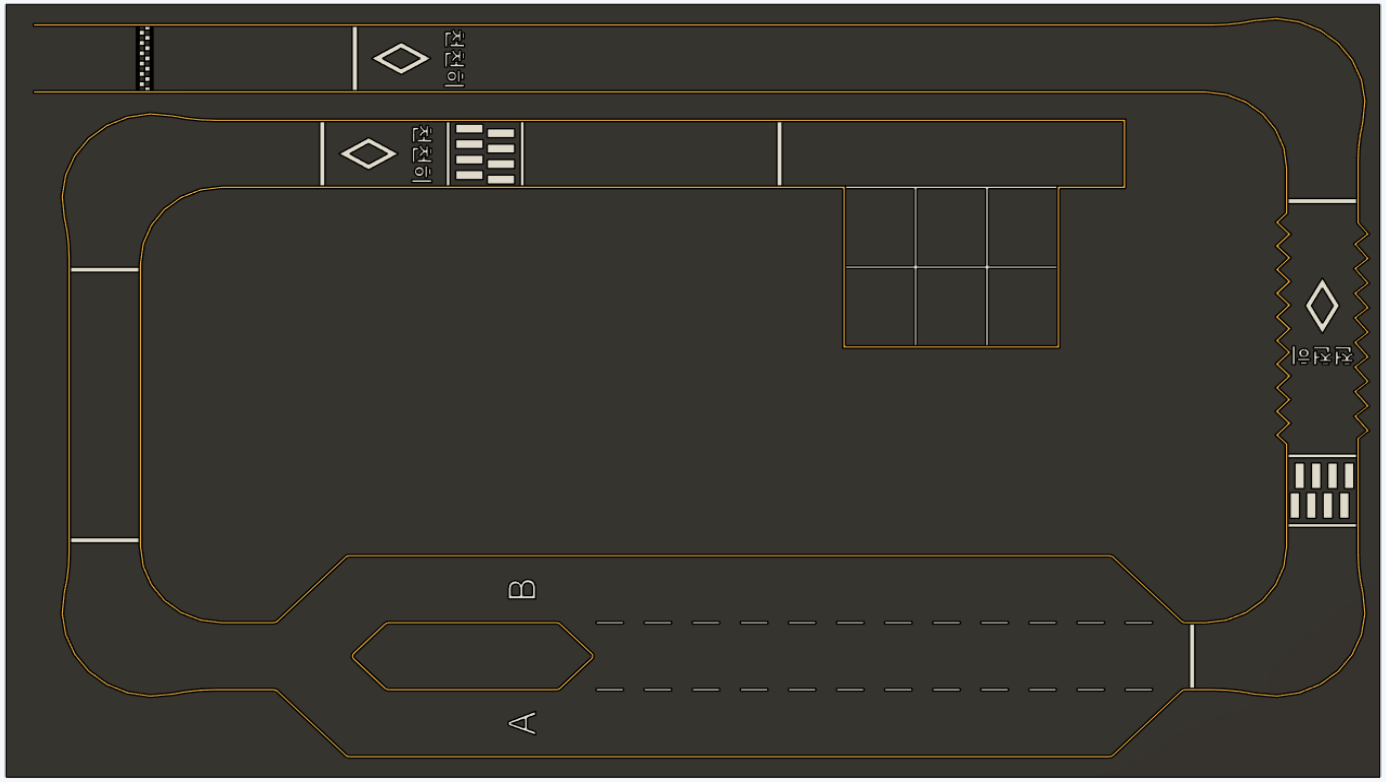
T 주차 : 차체 네 모서리가 선 안쪽

시설물 접촉 · 차선 이탈

한 바퀴 완주 / 10 분 제한

도면을 좌표와 카메라 화면으로 옮기기

첨부 CAD 도면의 선형을 사용합니다 . 시설물의 미확정 위치 · 동작은 학습용 설정입니다 .



10 m × 6 m

차로 폭 약 0.55 m

출발 → 서행 → 보행자 → 장애물 → 갈림길 → 라바콘 → 신호등 → T 주차

ROS 2 에서는 입력과 출력을 먼저 고정합니다

Gazebo 차량은 카메라 · 라이다 · 주행 상태를 발행하고 , 정답 코드는 센서로만 판단합니다 .

Gazebo → 정답 코드

/camera/image_raw · Image

/scan · LaserScan

/odom · Odometry 의 속도만 사용

시뮬레이션 시간 /clock

정답 코드 → Gazebo

/drive · AckermannDriveStamped

speed: m/s

steering_angle: rad, 좌회전 +

어댑터 : yaw_rate = v × tan(δ) / L

차선 주행은 작은 단계로 나눕니다

흰색 픽셀을 조향 명령으로 바로 바꾸기보다 , 중간 결과를 보며 튜닝합니다 .

흰색 HSV → 지면 투영 → Sliding window → Scan line → PID

- 수평 정지선과 노면 문자는 차선 후보에서 분리합니다 .
- 한쪽 차선만 보이면 좌우 정체성을 유지하고 법선 방향으로 중심을 복원합니다 .

• Scan line 목표점을 향한 기본 조향에 PID 보정을 더합니다 .

미션은 기본 주행의 목표와 속도를 바꿉니다

미션 순서 + 반복 인식 확인으로 옆 도로의 표지판이 잘못 작동하는 일을 줄입니다 .

카메라 / YOLOv8

서행 → 속도 상한 낮추기

보행자 → 정지선 접근 + 3 초 대기

좌 / 우 → 방향 기억 , 장애물 뒤 차로 선택

적색 · 황색 → 정지 , 녹색 → 발진

라이다 / 상태 관리

장애물 → 차체 폭을 반영한 빈 공간

라바콘 → 충돌 여유 + 안쪽 통과 조건

필요한 센서가 오래됨 → 정지

주차 중 장애물 · 라이다 / 속도 끊김 → 중단

정지선 : 보이는 위치와 실제 거리는 다릅니다

±5 cm 는 설정값만 넣는다고 충족되는 조건이 아닙니다 . 정지 오차를 반복 측정해야 합니다 .

- 카메라 높이 · 기울기 · 화각으로 픽셀을 지면 거리로 바꿉니다 .
- 거리 기준은 카메라가 아니라 앞바퀴입니다 .
- 가까워져 화면 밖으로 사라진 첫 정지선은 주행 속도로 추적합니다 .
- 다음 횡단보도 줄로 목표를 바꾸면 정지 위치가 밀립니다 .
- 보행자 대기 시간은 실제 속도가 0 에 가까워진 뒤부터 셉니다 .

T 주차 : 시간표 전에 진입 상태를 맞춥니다

하드코딩의 재현성은 출발 자세의 재현성에 달려 있습니다 .

① 영점 조절

주차 표지판으로 미션 진입

기준선까지 남은 거리 맞추기

차선 중심 오차 · 방향 오차 확인

정지 상태가 안정되면 시퀀스 시작

② YAML 시퀀스

시간 · 속도 · 조향각의 목록

전진 / 후진 / 조향 / 정지

장애물 감지 시 재개하지 않고 중단

정렬 뒤 영상 미사용 , 라이다 · 속도 감시

YOLOv8: rosbag 부터 학습까지 연결합니다

시뮬레이터에서 직접 기록한 이미지와 동기화된 라벨을 사용합니다 .

- record → rosbag: 카메라 + 시뮬레이터 정답 상자
- extract → 이미지 프레임 + YOLO 형식 라벨 + 데이터 분리
- train → 8 개 클래스 학습 → best.pt
  - 좌 / 우 의미가 바뀌는 좌우 반전 증강은 사용하지 않습니다 .
  - 신호 색이 바뀌는 강한 색상 증강도 피합니다 .
  - 주행 코드는 정답 상자를 읽지 않고 학습 모델로 추론합니다 .

검증한 것과 아직 검증하지 않은 것

2026-10-28 · 최종 기본 코스 + 수정 구간 검증

확인한 결과

최종 기본 코스 무개입 완주

13 개 체크포인트 ·9 개 기준 통과

카메라 2 초 중단 주차 시험 통과

정지선 ±5cm· 콘 4 개 안쪽 통과

77 개 테스트 ·YOLO mAP50 0.9888

추가로 확인할 결과

최종 소스의 우측 전체 재시험

여러 배치 · 조명에서의 반복 성공률

실물 LIMO 성능 · 공식 채점

복붙으로 실행하고 , YAML 하나로 조정합니다

프로젝트 README.md 에 전체 명령이 있습니다 . 사용자 실행 파일은 scripts/ 에 모았습니다 .

```
bash scripts/setup.sh configs/config.yaml


#  서로 다른 터미널에서 실행
bash scripts/simulator.sh configs/config.yaml
bash scripts/answer.sh configs/config.yaml
```

모델이 아직 없으면 record → extract → train 을 먼저 실행합니다 .

직접 돌려 보면서 고친 세 가지

구현 과정의 실제 실패 사례입니다 . 성공 여부는 앞의 검증 결과와 함께 확인하세요 .

차선 : 흰색도 역할이 다릅니다

노면 글자 · 마름모를 차선으로 오인

한 곡선의 두 봉우리를 양쪽 선으로 오인

연결 성분 + 이전 좌우 판단으로 수정

정답 데이터도 확인해야 합니다

red 라벨인데 켜진 등이 화면 밖

다른 표지판이 신호등을 가린 표본

켜진 등이 보이는 표본만 남기고 재촬영

주차 시간표도 실제 물리로 반복 보정

같은 카메라 프레임을 다시 쓰면 ?

전체 주행 시험에서 발견한 시간 처리 문제입니다 . 거리뿐 아니라 촬영 시각도 필요합니다 .

수정 전 : 정지선 6.51 cm 초과

묵은 RGB 의 거리를 매 tick 다시 대입

속도로 계산한 이동 거리가 되돌아감

기록을 대조해 약 10.5 cm 편향 확인

수정 후 : 정지선 2.20 cm 앞

새 프레임만 한 번 반영 + 촬영 후 이동 보정

처리가 늦으면 전진 거리를 제한

이 PC 에 맞춰 시뮬레이션 처리 여유 확보

실제 허용 범위 안에서 3.138 초 정지

회전 중에는 점선의 모양도 바뀝니다

하단 코너에서 장애물 회피로 넘어갈 때 발생한 실제 실패입니다 .

실패 : 잘못 기억한 도로 방향

회전하자 점선의 가로 / 세로 비율이 커짐

정상 점선을 지우고 합류선만 남김

잘못 저장한 방향으로 회피를 계속함

평지에서 시작한 구간 시험에는 없던 문제

수정 : 점선 여러 개로 확인

회전된 사각형의 짧은 변 / 긴 변 사용

여러 점선의 방향이 일치할 때만 채택

실제 진입 방향 오차 0.22° 로 개선

코너부터 장애물 · 갈림길 · 주차까지 재시험

같이 확인해 볼 실험

실패 장면을 기록하고 , 한 번에 한 가지 원인을 수정한 뒤 같은 조건으로 재시험합니다 .

- 차선 화면과 /answer/debug_image 를 나란히 비교하기
- 조향이 흔들리면 인식 오차와 PID 오차를 따로 확인하기
- 동일한 정지선에서 여러 번 멈추고 앞바퀴 오차 재기
- 주차 진입 자세를 바꿔 영점 조절이 복원되는지 보기
- 다른 조명 · 시점의 데이터로 YOLO 오검출 확인하기

참고 : 대회 규정 ·CAD / 2026-orda-education / unity-AD-simulator / ai-autonomous-driving-competition-2026